SLIP NO. 1

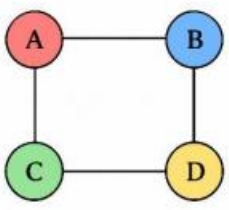
(1)

Q1) Write a program to implement Breadth First Search (BFS) for the Monkey Banana Problem.

```
from collections import deque
queue = deque(["Monkey at Door"])
visited = []
while queue:
    state = queue.popleft()
    if state not in visited:
        visited.append(state)
        print(state)
    if state == "Monkey at Door":
        queue.append("Monkey at Box")
    elif state == "Monkey at Box":
        queue.append("Box Under Banana")
    elif state == "Box Under Banana":
        queue.append("Monkey Climbs Box")
    elif state == "Monkey Climbs Box":
        queue.append("Monkey Gets Banana")
    elif state == "Monkey Gets Banana":
        print("\nGoal: Banana Obtained!")
        break
```

(2)

Q2) Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach.



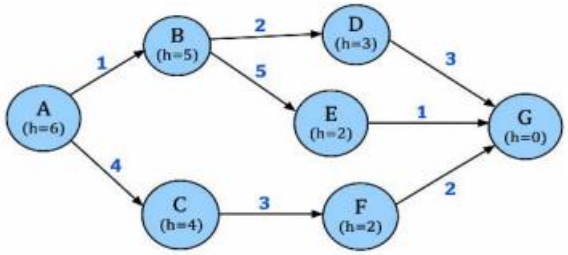
```
def is_safe(v, graph, color, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True
def graph_coloring(graph, m, color, v):
    if v == len(graph):
        return True
    for c in range(1, m + 1):
        if is_safe(v, graph, color, c):
            color[v] = c
            if graph_coloring(graph, m, color, v + 1):
                return True
            color[v] = 0
    return False
graph = [
    [0, 1, 1, 0],
    [1, 0, 0, 1],
    [1, 0, 0, 1],
    [0, 1, 1, 0],
]
m = 3
color = [0] * len(graph)
color_names = ["", "Red", "Green", "Blue"]
vertices = ["A", "B", "C", "D"]
if graph_coloring(graph, m, color, 0):
    print("Graph/Map Coloring Solution:")
    for i in range(len(vertices)):
        print(f"{vertices[i]} -> {color_names[color[i]]}")
else:
    print(f"No solution exists with {m} colors.")
```

OR

(2)

Q2) Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node

.




```
import heapq

def a_star_search(graph, heuristics, start, goal):
    open_set = []
    heapq.heappush(open_set, (heuristics[start], 0, start, [start]))
    g_costs = {start: 0}
    closed_set = set()
    steps = []
    while open_set:
        f_val, g_val, current, path = heapq.heappop(open_set)
        steps.append({
            'node': current,
            'g': g_val,
            'h': heuristics[current],
            'f': f_val,
            'path': path
        })
        if current == goal:
            return path, g_val, steps
        if current in closed_set:
            continue
        closed_set.add(current)
```

```
        for neighbor, edge_cost in graph.get(current, []):
            tentative_g = g_val + edge_cost
            if neighbor not in g_costs or tentative_g < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g
                f_score = tentative_g + heuristics[neighbor]
                heapq.heappush(open_set, (f_score, tentative_g, neighbor, path + [neighbor]))
    return None, float('inf'), steps
def main():
    graph = {
        'A': [('B', 1), ('C', 4)],
        'B': [('D', 2), ('E', 5)],
        'C': [('F', 3)],
        'D': [('G', 3)],
        'E': [('G', 1)],
        'F': [('G', 2)],
        'G': []
    }
    heuristics = {
        'A': 6,
        'B': 5,
        'C': 4,
        'D': 3,
        'E': 2,
        'F': 2,
        'G': 0
    }
    start_node = 'A'
    goal_node = 'G'
    print("          A* SEARCH ALGORITHM (SHORTEST PATH)          \n")
    print(f"Source Node:     {start_node} (h = {heuristics[start_node]})")
    print(f"Destination Node: {goal_node} (h = {heuristics[goal_node]})\n")
```

```
    print(f"{'Step':<5} | {'Node':<5} | {'g(n)':<6} | {'h(n)':<6} | {'f(n) = g + h':<14} | {'Path So Far'}")
    path, cost, steps = a_star_search(graph, heuristics, start_node, goal_node)
    for idx, s in enumerate(steps, 1):
        path_str = " -> ".join(s['path'])
        print(f"{idx:<5} | {s['node']:<5} | {s['g']:<6} | {s['h']:<6} | {s['f']:<14} | {path_str}")
    if path:
        print(f"\nShortest Path: {' -> '.join(path)}")
        print(f"Total Path Cost: {cost}")
    else:
        print("\nNo path found.")
if __name__ == "__main__":
    main()
```

SLIP NO. 2

Q.1)

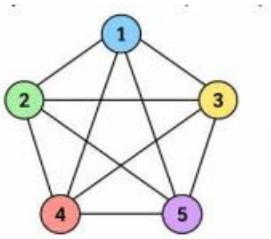
Q1) Write a program to solve the Water Jug Problem using BFS.

```
from collections import deque
def water_jug_bfs(capacity_a=4, capacity_b=3, target=2):
    visited = set()
    queue = deque([((0, 0), ["Initial State: (0, 0)"])])
    while queue:
        (x, y), path = queue.popleft()
        if x == target or y == target:
            return (x, y), path
        if (x, y) in visited:
            continue
        visited.add((x, y))
        operations = [
            ((capacity_a, y), f"Fill Jug A -> ({capacity_a}, {y})"),
            ((x, capacity_b), f"Fill Jug B -> ({x}, {capacity_b})"),
            ((0, y), f"Empty Jug A -> (0, {y})"),
            ((x, 0), f"Empty Jug B -> ({x}, 0)"),
            ((max(0, x - (capacity_b - y)), min(capacity_b, x + y)), f"Pour A into B -> ({max(0, x - (capacity_b - y))}, {min(capacity_b, x + y)})"),
            ((min(capacity_a, x + y), max(0, y - (capacity_a - x))), f"Pour B into A -> ({min(capacity_a, x + y)}, {max(0, y - (capacity_a - x))})"),
        ]
        for next_state, action in operations:
            if next_state not in visited:
                queue.append((next_state, path + [action]))
    return None, []
print("WATER JUG PROBLEM USING BFS (Capacity: A=4L, B=3L)\n")
final_state, steps = water_jug_bfs()
for i, step in enumerate(steps):
```

```
    print(f"Step {i}: {step}")
print(f"\nGoal Achieved! Exactly 2 liters measured in: {final_state}")
```

(2)

Q2) Write a program to implement the Backtracking Algorithm for solving a constraint satisfaction problem.



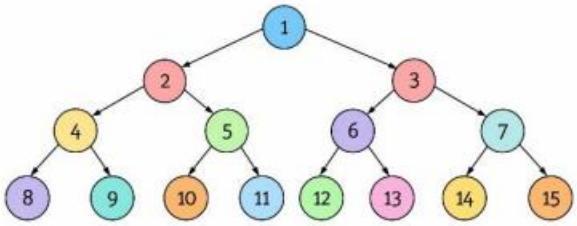

```
def is_safe(v, graph, color, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True
def solve_csp_backtracking(graph, m, color, v):
    if v == len(graph):
        return True
    for c in range(1, m + 1):
        if is_safe(v, graph, color, c):
            color[v] = c
            if solve_csp_backtracking(graph, m, color, v + 1):
                return True
            color[v] = 0
    return False
graph = [
    [0, 1, 1, 1, 1],
    [1, 0, 1, 1, 1],
    [1, 1, 0, 1, 1],
    [1, 1, 1, 0, 1],
    [1, 1, 1, 1, 0],
```

```
]
m = 5
color = [0] * len(graph)
color_names = ["", "Blue", "Green", "Yellow", "Red", "Purple"]
vertices = ["Node 1", "Node 2", "Node 3", "Node 4", "Node 5"]
print("CSP GRAPH COLORING USING BACKTRACKING (K5)")
if solve_csp_backtracking(graph, m, color, 0):
    print("Solution Found (Valid Color Assignment):")
    for i in range(len(vertices)):
        print(f"{vertices[i]} -> {color_names[color[i]]}")
else:
    print(f"No solution exists with {m} colors.")
```

SLIP NO. 3

(1)

Q1) Write a program to implement BFS traversal for a graph.




```
from collections import deque
def bfs_traversal(graph, start_node):
    visited = set()
    queue = deque([start_node])
    visited.add(start_node)
    traversal_order = []
    while queue:
        current = queue.popleft()
        traversal_order.append(current)
        for neighbor in graph.get(current, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return traversal_order
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [8, 9],
    5: [10, 11],
    6: [12, 13],
    7: [14, 15],
    8: [], 9: [], 10: [], 11: [],
```

```
    12: [], 13: [], 14: [], 15: []
}
start_node = 1
result = bfs_traversal(graph, start_node)
print("BREADTH FIRST SEARCH (BFS) GRAPH TRAVERSAL\n")
print(f"Starting Node: {start_node}\n")
print("BFS Traversal Order:\n")
print(" -> ".join(map(str, result)))
```

(2)

Q2) Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and NOT.

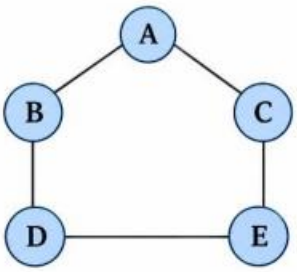
```
def logical_not(p):
    return not p
def logical_and(p, q):
    return p and q
def logical_or(p, q):
    return p or q
def logical_implies(p, q):
    return (not p) or q
print("PROPOSITIONAL LOGIC TRUTH TABLE\n")
print("P\t\tQ\t\tNOT P\tP AND Q\t\tP OR Q\t\tP -> Q")
values = [True, False]
for p in values:
    for q in values:
        not_p = logical_not(p)
        p_and_q = logical_and(p, q)
        p_or_q = logical_or(p, q)
        p_implies_q = logical_implies(p, q)
        print(f"{p}\t{q}\t{not_p}\t{p_and_q}\t\t{p_or_q}\t\t{p_implies_q}")
print("\nEVALUATING CUSTOM LOGICAL EXPRESSIONS\n")
```

```
P = True
Q = False
print(f"Given Values: P = {P}, Q = {Q}\n")
NOT = logical_not(P)
print(f"1. NOT P                : {NOT}")
AND = logical_and(P, Q)
print(f"2. P AND Q              : {AND}")
OR = logical_or(P, Q)
print(f"3. P OR Q               : {OR}")
```

SLIP NO. 4

(1)

Q1) Write a program to solve a state-space search problem using BFS.
Start Vertex = A




```
from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    traversal = []
    while queue:
        node = queue.popleft()
        traversal.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return traversal
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'E'],
    'D': ['B', 'E'],
    'E': ['C', 'D']
}
start_vertex = 'A'
```

```
result = bfs(graph, start_vertex)
print("BFS Traversal from Start Vertex A:\n")
print(" -> ".join(result))
```

(2)

Q2) Write a program to implement Predicate Logic using predicates, variables, and quantifier.

```
domain = ["Atharva", "Raha", "Pia"]
def is_human(x):
    return x in ["Atharva", "Raha", "Pia"]
def is_mortal(x):
    return is_human(x)
def is_student(x):
    return x in ["Atharva"]
all_mortal = all(is_mortal(x) for x in domain)
exists_student = any(is_student(x) for x in domain)
print("PREDICATE LOGIC EVALUATION\n")
print("Domain of Variables (x):", domain)
print("\nPredicate Evaluation:")
print("is_human('Atharva') :", is_human("Atharva"))
print("is_mortal('Atharva'):", is_mortal("Atharva"))
print("\nUniversal Quantifier (Mortal(x)):", all_mortal)
print("Existential Quantifier (Student(x)):", exists_student)
```

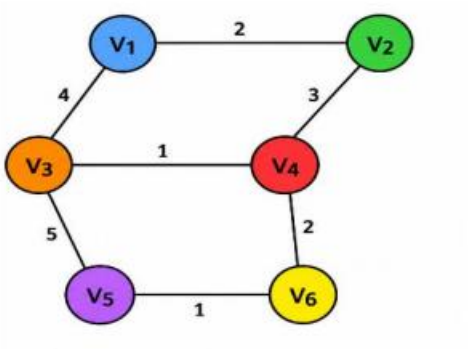
SLIP NO. 7

(1)

Q1) Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node.

Source = V_1

Destination = V_6




```
import heapq

graph = {
    'V1': [('V2', 2), ('V3', 4)],
    'V2': [('V1', 2), ('V4', 3)],
    'V3': [('V1', 4), ('V4', 1), ('V5', 5)],
    'V4': [('V2', 3), ('V3', 1), ('V6', 2)],
    'V5': [('V3', 5), ('V6', 1)],
    'V6': [('V4', 2), ('V5', 1)]
}

heuristics = {
    'V1': 6,
    'V2': 4,
    'V3': 3,
    'V4': 2,
    'V5': 1,
    'V6': 0
}

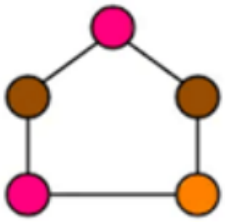
def a_star_search(graph, heuristics, start, goal):
    open_set = []
    initial_f = heuristics.get(start, 0)
    heapq.heappush(open_set, (initial_f, 0, start, [start]))
```

```
    g_scores = {start: 0}
    visited = set()
    print(f"{'Node Expanded':<15}{'g(n)':<10}{'h(n)':<10}{'f(n)':<10}{'Current Path'}")
    while open_set:
        f_score, neg_g, current, path = heapq.heappop(open_set)
        current_g = -neg_g
        print(f"{current:<15}{current_g:<10}{heuristics.get(current, 0):<10}{f_score:<10}{' ->
'.join(path)}")
        if current == goal:
            return path, current_g
        if current in visited:
            continue
        visited.add(current)
        for neighbor, weight in graph.get(current, []):
            if neighbor in visited:
                continue
            tentative_g = current_g + weight
            if tentative_g < g_scores.get(neighbor, float('inf')):
                g_scores[neighbor] = tentative_g
                h = heuristics.get(neighbor, 0)
                f = tentative_g + h
                new_path = path + [neighbor]
                heapq.heappush(open_set, (f, -tentative_g, neighbor, new_path))
    return None, float('inf')
if __name__ == "__main__":
    start_node = 'V1'
    goal_node = 'V6'
    print(f" A* Search from {start_node} to {goal_node} \n")
    shortest_path, total_cost = a_star_search(graph, heuristics, start_node, goal_node)
    if shortest_path:
        print(f"\nShortest Path: {' -> '.join(shortest_path)}")
```

```
    print(f"Total Path Cost: {total_cost}")
  else:
    print("No path found.")
```

(2)

Q2) Write a program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach.



```
class MapColoringCSP:
  def __init__(self, variables, domains, neighbors):
    self.variables = variables
    self.domains = domains
    self.neighbors = neighbors
    self.assignment = {}
  def is_consistent(self, var, color):
    for neighbor in self.neighbors.get(var, []):
      if neighbor in self.assignment and self.assignment[neighbor] == color:
        return False
    return True
  def backtrack(self):
    if len(self.assignment) == len(self.variables):
      return self.assignment
    unassigned_vars = [v for v in self.variables if v not in self.assignment]
    var = unassigned_vars[0]
    for color in self.domains[var]:
      if self.is_consistent(var, color):
        self.assignment[var] = color
```

```
                result = self.backtrack()
                if result is not None:
                    return result
                del self.assignment[var]
        return None
    def solve(self):
        self.assignment = {}
        return self.backtrack()
if __name__ == "__main__":
    variables = ['V1_Top', 'V2_Right', 'V3_BottomRight', 'V4_BottomLeft', 'V5_Left']
    colors = ['Pink', 'Brown', 'Orange']
    domains = {var: colors.copy() for var in variables}
    neighbors = {
        'V1_Top': ['V5_Left', 'V2_Right'],
        'V2_Right': ['V1_Top', 'V3_BottomRight'],
        'V3_BottomRight': ['V2_Right', 'V4_BottomLeft'],
        'V4_BottomLeft': ['V3_BottomRight', 'V5_Left'],
        'V5_Left': ['V4_BottomLeft', 'V1_Top']
    }
    csp = MapColoringCSP(variables, domains, neighbors)
    solution = csp.solve()
    print("   MAP COLORING PROBLEM (CSP BACKTRACKING SEARCH)   ")
    if solution:
        print("\nSolution Found Successfully:")
        print(f"{'Vertex / Region':<25} {'Assigned Color\n'}")
        for var, color in solution.items():
            print(f"{var:<25} {color}")
    else:
        print("No valid coloring exists for the given constraints.")
```

SLIP NO. 8

(1)

Q1) Write a program to solve the 4-Puzzle Problem using the A* Search Algorithm.

```
import heapq
class PuzzleNode:
    def __init__(self, state, parent=None, move="START", g=0, h=0):
        self.state = state
        self.parent = parent
        self.move = move
        self.g = g
        self.h = h
        self.f = g + h
        self.blank_pos = self.state.index(0)
    def __lt__(self, other):
        if self.f == other.f:
            return self.g > other.g
        return self.f < other.f
def manhattan_distance(state, goal):
    distance = 0
    for idx, tile in enumerate(state):
        if tile != 0:
            goal_idx = goal.index(tile)
            curr_r, curr_c = divmod(idx, 2)
            goal_r, goal_c = divmod(goal_idx, 2)
            distance += abs(curr_r - goal_r) + abs(curr_c - goal_c)
    return distance
def get_successors(node, goal):
    successors = []
    r, c = divmod(node.blank_pos, 2)
    directions = [
```

```
        (-1, 0, "UP"),
        (1, 0, "DOWN"),
        (0, -1, "LEFT"),
        (0, 1, "RIGHT")
    ]
    for dr, dc, move_name in directions:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 2 and 0 <= nc < 2:
            new_blank = nr * 2 + nc
            new_state = list(node.state)
            new_state[node.blank_pos], new_state[new_blank] = new_state[new_blank],
new_state[node.blank_pos]
            new_state_tuple = tuple(new_state)
            g = node.g + 1
            h = manhattan_distance(new_state_tuple, goal)
            successors.append(PuzzleNode(new_state_tuple, node, move_name, g, h))
    return successors
def a_star_4_puzzle(start_state, goal_state):
    open_list = []
    initial_h = manhattan_distance(start_state, goal_state)
    start_node = PuzzleNode(start_state, None, "START", 0, initial_h)
    heapq.heappush(open_list, start_node)
    closed_set = {}
    while open_list:
        current = heapq.heappop(open_list)
        if current.state == goal_state:
            path = []
            while current:
                path.append(current)
                current = current.parent
            return path[::-1]
```

```
        if current.state in closed_set and closed_set[current.state] <= current.g:
            continue
        closed_set[current.state] = current.g
        for succ in get_successors(current, goal_state):
            if succ.state in closed_set and closed_set[succ.state] <= succ.g:
                continue
            heapq.heappush(open_list, succ)
    return None
def print_board(state):
    for r in range(2):
        row = [str(state[r * 2 + c]) if state[r * 2 + c] != 0 else "_" for c in range(2)]
        print(" ".join(row))
    print()
if __name__ == "__main__":
    initial_state = (2, 0, 1, 3)
    goal_state = (1, 2, 3, 0)
    print("Initial State:")
    print_board(initial_state)
    print("Goal State:")
    print_board(goal_state)
    solution_path = a_star_4_puzzle(initial_state, goal_state)
    if solution_path:
        total_moves = len(solution_path) - 1
        print(f"Solution Found in {total_moves} move(s):\n")
        for step_idx, node in enumerate(solution_path):
            if step_idx == 0:
                print(f"Step 0: Initial Board (g={node.g}, h={node.h}, f={node.f})")
            else:
                print(f"Step {step_idx}: Move Blank {node.move} (g={node.g}, h={node.h}, f={node.f})")
            print_board(node.state)
        print(f"Optimal Total Moves: {total_moves}")
```

```
    else:
        print("No solution exists for the given configuration.")
```

(2)

Q2) Write a program to design a Rule-Based Expert System for simple decision-making.

```
temperature = float(input("Enter body temperature in (C): "))
cough = input("Does the patient have a cough? (yes/no): ").lower()
if temperature > 38:
    fever = True
else:
    fever = False
if fever and cough == "yes":
    disease = "Flu"
    recommendation = "Take rest, drink plenty of fluids, and consult a doctor."
elif fever:
    disease = "Fever"
    recommendation = "Monitor symptoms and consult a doctor if needed."
else:
    disease = "No major illness detected"
    recommendation = "Maintain a healthy lifestyle."
print("\nDiagnosis Report")
print("Fever:", fever)
print("Diagnosis:", disease)
print("Recommendation:", recommendation)
```

SLIP NO. 9

(1)

Q1) Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function.

```
def objective_function(x):
    return -(x**2) + (4 * x)
def hill_climbing(start_x, step_size=0.1, max_iterations=1000):
    current_x = start_x
    current_value = objective_function(current_x)
    for _ in range(max_iterations):
        neighbors = [current_x - step_size, current_x + step_size]
        best_neighbor = max(neighbors, key=objective_function)
        best_value = objective_function(best_neighbor)
        if best_value <= current_value:
            break
        current_x = best_neighbor
        current_value = best_value
    return round(current_x, 2), round(current_value, 2)
start = 0.0
max_x, max_val = hill_climbing(start)
print("HILL CLIMBING ALGORITHM\n")
print("Objective Function: f(x) = -x^2 + 4x")
print(f"Starting Point: x = {start}\n")
print(f"Optimal Point (x)  : {max_x}")
print(f"Maximum Value f(x) : {max_val}")
```

(2)

Q2) Write a program to implement the Forward Chaining inference mechanism.

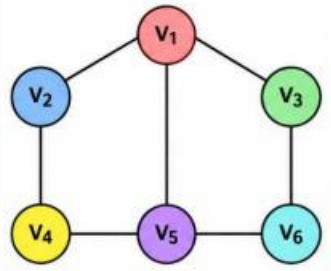
```
def forward_chaining(rules, facts, goal):
    inferred = set(facts)
```

```
    applied_rules = []
    while True:
        rule_applied = False
        for premises, conclusion in rules:
            if all(p in inferred for p in premises) and conclusion not in inferred:
                inferred.add(conclusion)
                applied_rules.append((premises, conclusion))
                rule_applied = True
                if conclusion == goal:
                    return True, applied_rules
        if not rule_applied:
            break
    return goal in inferred, applied_rules
rules = [
    (["High Temperature"], "Fever"),
    (["Fever", "Cough"], "Flu"),
    (["Flu"], "Rest and Medication")
]
initial_facts = ["High Temperature", "Cough"]
target_goal = "Rest and Medication"
success, steps = forward_chaining(rules, initial_facts, target_goal)
print("FORWARD CHAINING INFERENCE MECHANISM\n")
print(f"Initial Facts: {initial_facts}")
print(f"Target Goal  : {target_goal}\n")
print("Inference Steps:")
for premises, conclusion in steps:
    premise_str = " AND ".join(premises)
    print(f"IF {premise_str} THEN {conclusion}")
print(f"\nGoal Achieved: {success}")
```

SLIP NO. 14

(1)

Q1) Write a program to implement BFS traversal for a graph.




```
from collections import deque
graph = {
    'V1': ['V2', 'V3', 'V5'],
    'V2': ['V1', 'V4'],
    'V3': ['V1', 'V6'],
    'V4': ['V2', 'V5'],
    'V5': ['V1', 'V4', 'V6'],
    'V6': ['V3', 'V5']
}
def bfs_traversal(graph, start_node):
    visited_order = []
    visited_set = set()
    queue = deque([start_node])
    visited_set.add(start_node)
    while queue:
        current_node = queue.popleft()
        visited_order.append(current_node)
        for neighbor in sorted(graph[current_node]):
            if neighbor not in visited_set:
                visited_set.add(neighbor)
                queue.append(neighbor)
    return visited_order
if __name__ == "__main__":
```

```
start_vertex = 'V1'
result = bfs_traversal(graph, start_vertex)
print(f"Starting Vertex: {start_vertex}")
print("BFS Traversal Order:")
print(" -> ".join(result))
```

(2)

Q2) Write a program or report to compare Supervised Learning and Unsupervised Learning techniques.

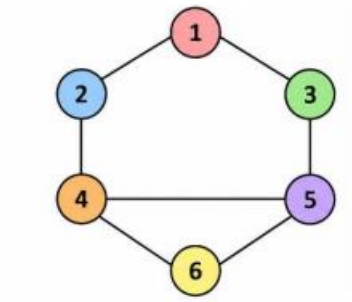
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
print("1. SUPERVISED LEARNING DEMONSTRATION")
X_supervised = [
    [1, 1],
    [1, 0],
    [0, 1],
    [0, 0]
]
y_supervised = [1, 0, 0, 0]
dt_model = DecisionTreeClassifier()
dt_model.fit(X_supervised, y_supervised)
test_applicant = [[1, 1]]
supervised_pred = dt_model.predict(test_applicant)
decision = "Loan Approved" if supervised_pred[0] == 1 else "Loan Rejected"
print(f"Test Applicant Features: {test_applicant[0]}")
print(f"Supervised Model Prediction: {decision}\n")
print("2. UNSUPERVISED LEARNING DEMONSTRATION")
X_unsupervised = [
    [15, 39],
    [15, 81],
    [16, 6],
```

```
    [16, 77],
    [17, 40],
    [17, 76]
]
kmeans = KMeans(n_clusters=2, random_state=42, n_init='auto')
cluster_labels = kmeans.fit_predict(X_unsupervised)
print("Customer Samples:")
for i, sample in enumerate(X_unsupervised):
    print(f"Customer {i+1} {sample} assigned to Cluster {cluster_labels[i]}")
```

SLIP NO. 15

(1)

Q1) Write a program to solve a state-space search problem using DFS.
Start vertex - 1




```
def dfs(graph, node, visited=None, traversal=None):
    if visited is None:
        visited = set()
    if traversal is None:
        traversal = []
    visited.add(node)
    traversal.append(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, traversal)
    return traversal
graph = {
    1: [2, 3],
    2: [1, 4],
    3: [1, 5],
    4: [2, 5, 6],
    5: [3, 4, 6],
    6: [4, 5]
}
start_vertex = 1
result = dfs(graph, start_vertex)
print("DFS Traversal from Start Vertex 1:\n")
```

```
print(" -> ".join(map(str, result)))
```

(2)

Q2) Write a program to build a Knowledge Base using logical rules and facts.

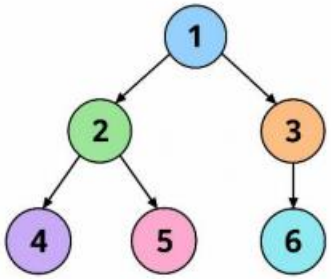
```
class KnowledgeBase:
    def __init__(self):
        self.facts = set()
        self.rules = []
    def add_fact(self, fact):
        self.facts.add(fact)
    def add_rule(self, conditions, conclusion):
        self.rules.append((conditions, conclusion))
    def infer(self):
        while True:
            new_fact_derived = False
            for conditions, conclusion in self.rules:
                if all(c in self.facts for c in conditions) and conclusion not in self.facts:
                    self.facts.add(conclusion)
                    new_fact_derived = True
            if not new_fact_derived:
                break
    def query(self, statement):
        return statement in self.facts
kb = KnowledgeBase()
kb.add_fact("has_fever")
kb.add_fact("has_cough")
kb.add_rule(["has_fever", "has_cough"], "has_flu")
kb.add_rule(["has_flu"], "needs_medicine")
print("KNOWLEDGE BASE SYSTEM\n")
print("Initial Facts:", sorted(list(kb.facts)))
```

```
kb.infer()
print("\nDerived Facts after Inference:", sorted(list(kb.facts)))
print("\nQueries:")
print("Is 'has_flu' True?       :", kb.query("has_flu"))
print("Is 'needs_medicine' True?:", kb.query("needs_medicine"))
print("Is 'has_headache' True?  :", kb.query("has_headache"))
```

SLIP NO. 16

(1)

Q1) Write a program to implement Depth Limited Search (DLS).




```
def dls(node, goal, limit, path):
    path.append(node)
    if node == goal:
        return True
    if limit <= 0:
        return False
    for child in graph[node]:
        if dls(child, goal, limit - 1, path):
            return True
    return False
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [],
    5: [],
    6: []
}
start_node = 1
target_node = 6
depth_limit = 2
path = []
```

```
found = dls(start_node, target_node, depth_limit, path)
print("DEPTH LIMITED SEARCH (DLS)\n")
print(f"Start Node  : {start_node}")
print(f"Target Node : {target_node}")
print(f"Depth Limit : {depth_limit}\n")
if found:
    print(f"Target {target_node} found within depth limit {depth_limit}!")
    print("Search Path :", " -> ".join(map(str, path)))
else:
    print(f"Target {target_node} not found within depth limit {depth_limit}.")
```

(2)

Q2) Write a program to implement the Backward Chaining inference mechanism.

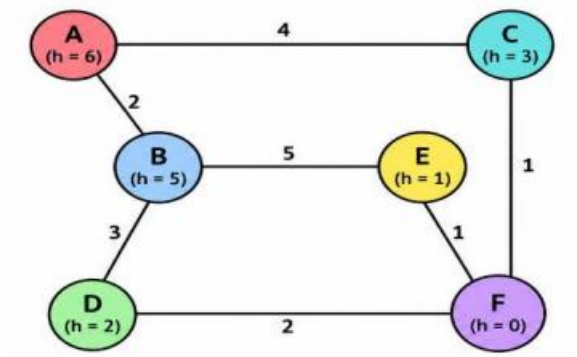
```
def backward_chaining(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()
    print(f"Checking Goal : '{goal}'")
    if goal in facts:
        print(f"-> '{goal}' is a known fact.")
        return True
    if goal in visited:
        return False
    visited.add(goal)
    for premises, conclusion in rules:
        if conclusion == goal:
            print(f"-> Matching Rule found: IF {' AND '.join(premises)} THEN {conclusion}\n")
            if all(backward_chaining(p, facts, rules, visited) for p in premises):
                print(f"-> Sub-goals satisfied, '{goal}' is proved.\n")
                return True
    return False
```

```
rules = [
    (["High Temperature"], "Fever"),
    (["Fever", "Cough"], "Flu")]
facts = {“High Temperature”, “Cough”}
target_goal = “Flu”
print(“BACKWARD CHAINING INFERENCE MECHANISM\n”)
print(f”Known Facts: {sorted(list(facts))}”)
print(f”Target Goal: {target_goal}\n”)
success = backward_chaining(target_goal, facts, rules)
print(f”Final Result: Goal ‘{target_goal}’ Proved = {success}”)
```

SLIP NO. 17

(1)

Q1) Write a program to implement the Best First Search Algorithm using heuristic values to find a path from a start node to a goal node.




```
from queue import PriorityQueue
def best_first_search(graph, heuristics, start, goal):
    visited = set()
    pq = PriorityQueue()
    pq.put((heuristics[start], start, [start]))
    while not pq.empty():
        h, current, path = pq.get()
        if current in visited:
            continue
        visited.add(current)
        if current == goal:
            return path
        for neighbor in graph[current]:
            if neighbor not in visited:
                pq.put((heuristics[neighbor], neighbor, path + [neighbor]))
    return None
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
```

```
    'D': ['B', 'F'],
    'E': ['B', 'F'],
    'F': ['C', 'D', 'E']
}
heuristics = {
    'A': 6,
    'B': 5,
    'C': 3,
    'D': 2,
    'E': 1,
    'F': 0
}
start_node = 'A'
goal_node = 'F'
path = best_first_search(graph, heuristics, start_node, goal_node)
print("BEST FIRST SEARCH ALGORITHM\n")
print(f"Start Node: {start_node}")
print(f"Goal Node : {goal_node}\n")
if path:
    print("Path found using Best First Search:")
    print(" -> ".join(path))
else:
    print("No path found.")
```

(2)

Q2) Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and IMPLIES.

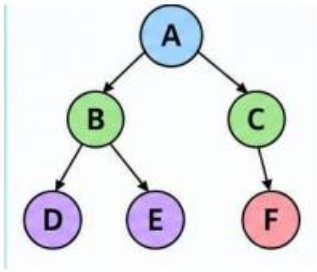
```
def logical_and(p, q):
    return p and q
def logical_or(p, q):
    return p or q
```

```
def logical_implies(p, q):
    return (not p) or q
print("TRUTH TABLE\n")
print("P\t\tQ\t\tP AND Q\t\tP OR Q\t\tP -> Q")
for p in [True, False]:
    for q in [True, False]:
        print(f"{p}\t{q}\t{logical_and(p, q)}\t\t{logical_or(p, q)}\t\t{logical_implies(p, q)}")
P = True
Q = False
print(f"\nGiven: P = {P}, Q = {Q}\n")
print("P AND Q  :", logical_and(P, Q))
print("P OR Q   :", logical_or(P, Q))
print("P -> Q   :", logical_implies(P, Q))
```

SLIP NO. 21

(1)

Q1) Write a program implement Iterative Deepening Search (IDS).

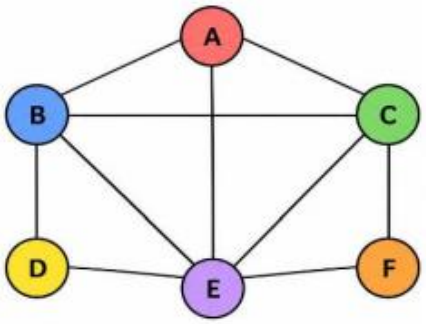



```
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
def depth_limited_search(node, goal, limit, visited_nodes):
    visited_nodes.append(node)
    if node == goal:
        return True
    if limit <= 0:
        return False
    for child in tree.get(node, []):
        if depth_limited_search(child, goal, limit - 1, visited_nodes):
            return True
    return False
def iterative_deepening_search(start_node, goal_node, max_depth):
    print(f"Starting Iterative Deepening Search (IDS)")
    print(f"Start Node: {start_node}, Target Goal: {goal_node}\n")
    for current_limit in range(max_depth + 1):
        visited_nodes = []
```

```
        found = depth_limited_search(start_node, goal_node, current_limit, visited_nodes)
        traversal_path = " -> ".join(visited_nodes)
        print(f"Iteration at Depth Limit {current_limit}:")
        print(f"Nodes visited: {traversal_path}")
        if found:
            print(f"Goal node '{goal_node}' found successfully at depth {current_limit}!\n")
            return True
        else:
            print(f"Goal not found at depth limit {current_limit}.\n")
    print(f"Goal node '{goal_node}' not found within maximum depth {max_depth}.")
    return False
if __name__ == "__main__":
    start = 'A'
    goal = 'F'
    max_search_depth = 3
    iterative_deepening_search(start, goal, max_search_depth)
```

(2)

Q2) Write a program to solve the Map Colouring Problem using the Constraint Satisfaction Problem (CSP) approach.




```
def is_safe(v, graph, color, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True
```

```
def graph_coloring(graph, m, color, v):
    if v == len(graph):
        return True
    for c in range(1, m + 1):
        if is_safe(v, graph, color, c):
            color[v] = c
            if graph_coloring(graph, m, color, v + 1):
                return True
            color[v] = 0
    return False
graph = [
    [0, 1, 1, 0, 1, 0],
    [1, 0, 1, 1, 1, 0],
    [1, 1, 0, 0, 1, 1],
    [0, 1, 0, 0, 1, 0],
    [1, 1, 1, 1, 0, 1],
    [0, 0, 1, 0, 1, 0]
]
m = 4
color = [0] * len(graph)
color_names = ["", "Red", "Blue", "Green", "Yellow"]
vertices = ["A", "B", "C", "D", "E", "F"]
if graph_coloring(graph, m, color, 0):
    print("Graph Coloring Solution (CSP Backtracking):\n")
    for i in range(len(vertices)):
        print(f"Vertex {vertices[i]} -> {color_names[color[i]]}")
else:
    print(f"No valid coloring exists with {m} colors.")
```

SLIP NO. 22

(1)

Q1) Write a program to solve the Tower of Hanoi Problem using state-space representation.

```
def print_state(pol):
  print("A:", pol["A"])
  print("B:", pol["B"])
  print("C:", pol["C"])
def move_disk(pol, start, end):
  disk = pol[start].pop()
  pol[end].append(disk)
  print(f"\nMove disk {disk} from {start} -> {end}")
  print_state(pol)
def tower_of_hanoi(n, start, helper, end, pol):
  if n  == 1:
    move_disk(pol, start, end)
    return
  tower_of_hanoi(n - 1, start, end, helper, pol)
  move_disk(pol, start, end)
  tower_of_hanoi(n - 1, helper, start, end, pol)
n = int(input("Enter the number of disks:"))
pol = {
  'A' : list(range(n, 0, -1)),
  'B' : [],
  'C' : []
}
print("Inititial State:")
print_state(pol)
tower_of_hanoi(n, 'A', 'B', 'C', pol)
print("Goal State Reached!")
```

(2)

Q2) Write a program to solve the N-Queens Problem using the Backtracking Algorithm.

```
def print_board(board):
    for row in board:
        print(" ".join("Q" if cell == 1 else "." for cell in row))
def is_safe(board, row, col, n):
    for i in range(col):
        if board[row][i] == 1:
            return False
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    i, j = row, col
    while i < n and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
    return True
def solve_n_queens(board, col, n):
    if col >= n:
        return True
    for row in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1
            if solve_n_queens(board, col + 1, n):
                return True
```

```
            board[row][col] = 0
    return False
N = int(input("Enter number of Queens (N): "))
board = [[0] * N for _ in range(N)]
print(f"\nN-QUEENS PROBLEM USING BACKTRACKING (N = {N})\n")
if solve_n_queens(board, 0, N):
    print("Solution Found:\n")
    print_board(board)
else:
    print("No solution exists.")
```

SLIP NO. 23

(1)

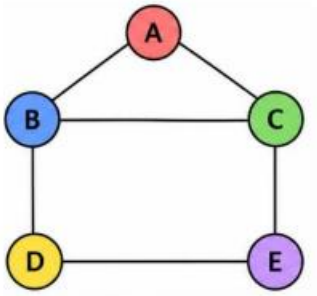
Q1) Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function. (F(x) = -x^2 + 4x)

```
def objective_function(x):
    return -x**2 + 4*x
def hill_climbing(start_x, step_size=0.5, max_iterations=50):
    current_x = start_x
    current_val = objective_function(current_x)
    print(f"Initial State: x = {current_x:.2f}, F(x) = {current_val:.4f}\n")
    for iteration in range(1, max_iterations + 1):
        left_neighbor = current_x - step_size
        right_neighbor = current_x + step_size
        left_val = objective_function(left_neighbor)
        right_val = objective_function(right_neighbor)
        if left_val > right_val:
            best_neighbor = left_neighbor
            best_val = left_val
        else:
            best_neighbor = right_neighbor
            best_val = right_val
        if best_val <= current_val:
            print(f"Stopping: Peak reached at iteration {iteration - 1}")
            break
        current_x = best_neighbor
        current_val = best_val
        print(f"Iteration {iteration}: Move to x = {current_x:.2f}, F(x) = {current_val:.4f}")
    return current_x, current_val
if __name__ == "__main__":
    initial_point = 0.0
    step = 0.5
```

```
print("Hill Climbing Algorithm for Optimization")
print("Function: F(x) = -x^2 + 4x\n")
optimal_x, max_value = hill_climbing(start_x=initial_point, step_size=step)
print("\nOptimization Result:")
print(f"Optimal x: {optimal_x:.2f}")
print(f"Maximum Value F(x): {max_value:.4f}")
```

(2)

Q2) Write a program to solve the Map Colouring Problem using the Constraint Satisfaction Problem (CSP) approach.



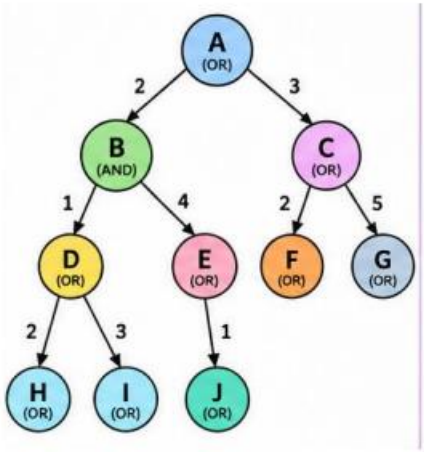

```
def is_safe(v, graph, color, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True
def graph_coloring(graph, m, color, v):
    if v == len(graph):
        return True
    for c in range(1, m + 1):
        if is_safe(v, graph, color, c):
            color[v] = c
            if graph_coloring(graph, m, color, v + 1):
                return True
            color[v] = 0
    return False
```

```
graph = [
    [0, 1, 1, 0, 0],
    [1, 0, 1, 1, 0],
    [1, 1, 0, 0, 1],
    [0, 1, 0, 0, 1],
    [0, 0, 1, 1, 0]
]
m = 3
color = [0] * len(graph)
color_names = ["", "Red", "Green", "Blue"]
vertices = ["A", "B", "C", "D", "E"]
if graph_coloring(graph, m, color, 0):
    print("Graph Coloring Solution (CSP Backtracking):\n")
    for i in range(len(vertices)):
        print(f"Vertex {vertices[i]} -> {color_names[color[i]]}")
else:
    print(f"No valid coloring exists with {m} colors.")
```

SLIP NO. 25

(1)

Q1) Write the program to implement the AO* Algorithm




```
class AOStar:
    def __init__(self, graph, heuristics):
        self.graph = graph
        self.H = heuristics
        self.solution = {}
    def get_min_cost_branch(self, node):
        if node not in self.graph or not self.graph[node]:
            return self.H.get(node, 0), []
        min_cost = float('inf')
        best_branch = []
        for branch in self.graph[node]:
            branch_cost = sum(cost + self.H.get(child, 0) for child, cost in branch)
            if branch_cost < min_cost:
                min_cost = branch_cost
                best_branch = [child for child, _ in branch]
        return min_cost, best_branch
    def solve(self, nodes_order):
        for node in nodes_order:
            min_cost, best_branch = self.get_min_cost_branch(node)
```

```
            self.H[node] = min_cost
            self.solution[node] = best_branch
    def print_solution(self, node):
        if node in self.solution and self.solution[node]:
            children = self.solution[node]
            print(f"{node} -> {' AND '.join(children)}")
            for child in children:
                self.print_solution(child)
graph = {
    'A': [
        [('B', 2)],
        [('C', 3)]
    ],
    'B': [
        [('D', 1), ('E', 4)]
    ],
    'C': [
        [('F', 2)],
        [('G', 5)]
    ],
    'D': [
        [('H', 2)],
        [('I', 3)]
    ],
    'E': [
        [('J', 1)]
    ],
    'F': [], 'G': [], 'H': [], 'I': [], 'J': []
}
heuristics = {
    'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0,
```

```
    'F': 0, 'G': 0, 'H': 0, 'I': 0, 'J': 0
}
ao = AOStar(graph, heuristics)
ao.solve(['D', 'E', 'B', 'C', 'A'])
print("AO* ALGORITHM IMPLEMENTATION\n")
print(f"Optimal Cost of Root Node A: {ao.H['A']}\n")
print("Optimal Solution Path (Marked Nodes):")
ao.print_solution('A')
```

(2)

Q2) Write a program to implement the Forward Chaining inference mechanism.

```
def forward_chaining(rules, facts, goal):
    inferred = set(facts)
    applied_rules = []
    while True:
        rule_applied = False
        for premises, conclusion in rules:
            if all(p in inferred for p in premises) and conclusion not in inferred:
                inferred.add(conclusion)
                applied_rules.append((premises, conclusion))
                rule_applied = True
                if conclusion == goal:
                    return True, applied_rules
        if not rule_applied:
            break
    return goal in inferred, applied_rules
rules = [
    (["High Temperature"], "Fever"),
    (["Fever", "Cough"], "Flu"),
    (["Flu"], "Rest and Medication")
```

```
]
initial_facts = ["High Temperature", "Cough"]
target_goal = "Rest and Medication"
success, steps = forward_chaining(rules, initial_facts, target_goal)
print("FORWARD CHAINING INFERENCE MECHANISM\n")
print(f"Initial Facts: {initial_facts}")
print(f"Target Goal  : {target_goal}\n")
print("Inference Steps:")
for premises, conclusion in steps:
    premise_str = " AND ".join(premises)
    print(f"IF {premise_str} THEN {conclusion}")
print(f"\nGoal Achieved: {success}")
```